

DNS Tunneling Detection System

Day 1 Project Initialization, Environment & Architectural Overview

PROJECT PHASE	PYTHON VERSION	TARGET PLATFORM	OVERALL STATUS
Day 1: Setup & Verification	Python 3.12.10 (64-bit)	Windows (Local Virtualenv)	100% PASS

1. EXECUTIVE SUMMARY

DNS Tunneling is an advanced cyber-attack technique that abuses the Domain Name System (DNS, UDP/TCP port 53) to exfiltrate confidential enterprise data or transmit malicious Command & Control (C2) payloads through standard network firewalls. Because DNS queries are critical for standard internet browsing and network discovery, port 53 is almost universally permitted outward, making it an attractive covert channel for adversaries.

The objective of this research project is to develop an **end-to-end Machine Learning pipeline** that distinguishes benign domain traffic from malicious DNS tunneling queries by analyzing lexical structure, statistical character distributions, and **Shannon Entropy**.

Day 1 Objective: Establish a robust, reproducible, and isolated development environment, construct the modular software architecture, configure version control, and confirm that all necessary scientific libraries function without error before building model components.

2. DAY 1 MILESTONES & ACCOMPLISHMENTS

- **Python 3.12 Installation:** Verified the absence of Python 3.12, obtained explicit authorization, and performed a clean installation of **Python 3.12.10 (64-bit)**.
- **Dedicated Workspace:** Initialized the root project directory on the Desktop:
C:\Users\arath\Desktop\DNS-Tunneling-Detection.
- **Virtual Environment (venv):** Created an isolated Python 3.12 virtual environment to guarantee dependency isolation and prevent global system package conflicts.
- **Package Installation:** Successfully resolved and installed all primary data science, machine learning, and explainability libraries via `pip`.
- **Modular Architecture (src/):** Created 8 distinct Python modules with clear academic docstrings defining the upcoming machine learning lifecycle.
- **Git Version Control:** Initialized a local Git repository and created the baseline commit (c9d7e48), specifically ensuring that the `data/` folder is preserved for dataset placement.
- **Automated Verification:** Created and executed `main.py`, testing environment variables, package imports, module syntax, and directory layout with zero errors.

3. PROJECT DIRECTORY ARCHITECTURE

The workspace adheres to standard academic and industry ML engineering practices:

```
DNS-Tunneling-Detection/
├─ data/                # Raw headerless datasets (training.csv & validating.csv)
├─ src/                 # Core pipeline modules
│   ├── __init__.py     # Package initialization
│   ├── data_loader.py  # Reads headerless CSVs into pandas DataFrames
│   ├── preprocessing.py # Cleans, normalizes, and validates domain strings
│   ├── feature_extraction.py # Extracts lexical & character distribution metrics
│   ├── entropy.py      # Computes Shannon Entropy (degree of payload randomness)
│   ├── train_models.py # Trains classification models (Random Forest, etc.)
│   ├── evaluate.py     # Computes Precision, Recall, F1, ROC-AUC & Confusion Matrix
│   └─ explainability.py # Generates SHAP explainable AI interpretability plots
├─ models/              # Serialized model pipelines (.joblib)
├─ results/             # Generated evaluation charts, metrics, and figures
├─ notebooks/           # Interactive Jupyter exploration notebooks
├─ venv/                # Isolated Python 3.12.10 virtual environment
├─ main.py              # Day 1 verification entry point & diagnostic script
├─ requirements.txt     # Version-locked dependency manifest
├─ README.md            # Academic documentation and architecture reference
└─ .gitignore           # Tracks data/ while excluding large binaries & venv
```

4. MODULAR PIPELINE BREAKDOWN (SRC/)

Module Name	Future Responsibility	Cybersecurity / Academic Value
data_loader.py	Ingests training.csv (15,000) and validating.csv (5,000).	Loads headerless data, maps Column 0 to Label and Column 1 to Domain name.
preprocessing.py	Normalizes domains (lowercasing, trimming root dots, validating strings).	Ensures clean, robust text inputs and eliminates corrupted entries.
feature_extraction.py	Extracts domain lengths, subdomain counts, digit ratios, consonant-vowel ratios.	Translates raw domain queries into numerical features for machine learning.
entropy.py	Computes Shannon Entropy : $\sum P(x) \log_2 P(x)$.	Key discriminator: Encoded tunneling payloads show high entropy vs. benign text.
train_models.py	Trains classifiers (Random Forest, Logistic Regression) & saves via joblib.	Evaluates ensemble learning effectiveness against adversarial evasions.
evaluate.py	Calculates Accuracy, Precision, Recall, F1-Score, and Confusion Matrix.	Prioritizes high Recall (catching threats) and low False Positives.
explainability.py	Computes SHAP (SHapley Additive exPlanations) values and plots.	Defeats the "black-box" dilemma by demonstrating why an alert fired.

5. DATASET SPECIFICATIONS (HEADERLESS)

The model will be trained and evaluated on two dedicated, headerless datasets situated inside the data/ directory:

Dataset Name	Record Count	Columns	Column 0 (Ground Truth Label)	Column 1 (Feature String)
training.csv	15,000 records	2 columns (Headerless)	0 = Regular / Benign Domain 1 = DNS Tunneling Attack	Domain query string (e.g., example.com)
validating.csv	5,000 records	2 columns (Headerless)	0 = Regular / Benign Domain 1 = DNS Tunneling Attack	Domain query string (e.g., hexpayload.tunnel.xyz)

Academic Note: Keeping the datasets strictly headerless avoids manual data tampering. The data_loader.py script programmatically assigns column headers (['label', 'domain']) during ingestion.

6. INSTALLED SOFTWARE ENVIRONMENT

All dependencies have been successfully installed inside venv and documented in `requirements.txt`:

Library	Version	Purpose in Project
Python	3.12.10	Base runtime interpreter
pandas	3.0.5	Data manipulation & tabular analysis
numpy	2.5.3	Numerical computing & vector arrays
scikit-learn	1.9.0	ML algorithms, cross-validation, and metrics
shap	0.52.0	Explainable AI (feature importance & reasoning)
matplotlib	3.11.1	Foundational plotting engine
seaborn	0.13.2	Statistical visualization & heatmaps
joblib	1.6.0	Model serialization and disk persistence

7. VERIFICATION & DIAGNOSTIC RESULTS

The verification script (main.py) was executed inside the virtual environment. All checks passed with 100% conformity:

```
=====
      DNS TUNNELING DETECTION - DAY 1 SETUP VERIFICATION
=====
1. Python Environment Check
  Python Version:      3.12.10
  Executable Path:     ...\DNS-Tunneling-Detection\venv\Scripts\python.exe
  In Virtual Env:      [YES] (venv active)
  Status:              [PASS] Compatible Python 3.12 detected.

2. Required Packages Check
- pandas              : [INSTALLED] (v3.0.5)
- numpy               : [INSTALLED] (v2.5.3)
- scikit-learn        : [INSTALLED] (v1.9.0)
- shap                : [INSTALLED] (v0.52.0)
- matplotlib          : [INSTALLED] (v3.11.1)
- seaborn             : [INSTALLED] (v0.13.2)
- joblib              : [INSTALLED] (v1.6.0)

3. Project Directory Structure Check
  Checking Directories:
    [OK] data/   [OK] src/   [OK] models/   [OK] results/   [OK] notebooks/
  Checking Files:
    [OK] requirements.txt  [OK] README.md   [OK] .gitignore   [OK] main.py
    [OK] 8 Python modules in src/

4. Source Modules Import & Syntax Check
  [OK] src.data_loader      (valid syntax)
  [OK] src.preprocessing    (valid syntax)
  [OK] src.feature_extraction (valid syntax)
  [OK] src.entropy          (valid syntax)
  [OK] src.train_models     (valid syntax)
  [OK] src.evaluate         (valid syntax)
  [OK] src.explainability   (valid syntax)

Summary & Verification Verdict:
  [SUCCESS] All Day 1 setup verification checks PASSED!
  The project environment and architecture are ready for Day 2.
=====
```

8. FULL PROJECT ROADMAP (6-DAY PLAN)

Phase	Milestone Name	Key Deliverables	Status
Day 1	Environment & Setup	Python 3.12 venv, directory architecture, Git repo, Day 1 verification.	COMPLETED
Day 2	Data Loading & Prep	Read training.csv (15k) & validating.csv (5k), check balance, clean strings.	<i>Upcoming</i>

Phase	Milestone Name	Key Deliverables	Status
Day 3	Feature Engineering	Compute Shannon Entropy, domain lengths, subdomain counts, character ratios.	<i>Upcoming</i>
Day 4	Model Training	Train Random Forest & Logistic Regression models; serialize with <code>joblib</code> .	<i>Upcoming</i>
Day 5	Model Evaluation	Assess Accuracy, Precision, Recall, ROC-AUC, generate Confusion Matrix plot.	<i>Upcoming</i>
Day 6	Explainability (SHAP)	Compute SHAP global and local importance plots to explain classifications.	<i>Upcoming</i>